

Java Language Internals & Concurrency — Interview Guide

The "do you actually know Java?" round, with examples and diagrams (Java 8+)

Target level: Mid-level software engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

Why this guide exists

Plenty of candidates can solve a LeetCode Medium but freeze when the interviewer pivots to "*what happens inside a HashMap when it resizes?*" or "*what does `volatile` actually guarantee?*" These questions separate people who **use** Java from people who **understand** it — and at Microsoft, Agoda, and UiPath they're a common part of the loop.

This guide has two parts:

- **Part A — Language & Runtime Depth.** How the collections, the memory model, generics, and the object contract really work under the hood.
- **Part B — Concurrency & Multithreading.** The threading model, the memory-visibility rules, locks vs atomics, and the classic thread-coordination problems you may be asked to code live.

Every topic follows the same shape: **the concept in plain language → a diagram → a code example → the traps interviewers probe → a crisp "what to say."** All code is Java 8+.

What's inside

#	Part A — Language & Runtime	What it proves
1	Collections internals (HashMap, ConcurrentHashMap)	You understand hashing, buckets, resizing
2	equals() / hashCode() contract	You know why broken hashing corrupts sets/maps
3	JVM memory & Garbage Collection	You understand heap/stack, GC roots, generations
4	Generics & type erasure	You know compile-time vs runtime types
5	Exceptions, try-with-resources, Optional	You write robust, leak-free code
6	Immutability & the String pool	You know why String is special and immutability is safe
7	Lambdas, Streams, CompletableFuture	You write modern functional Java

#	Part B — Concurrency	What it proves
8	Threads: lifecycle & creation	You know the basics cold
9	synchronized, volatile, happens-before	You understand the Java Memory Model
10	Locks & atomic variables	You pick the right tool for contention
11	Producer–Consumer (BlockingQueue)	You can code the canonical pattern
12	Thread-safe Singleton	You know double-checked locking + why volatile
13	Executors & thread pools	You don't spawn raw threads in real code
14	Latches, barriers, semaphores & deadlock	You can coordinate threads and avoid pitfalls

How to use it in the room

1. **Answer the concept first, in one sentence.** Then go deeper if they want.
 2. **Draw the picture.** "A HashMap is an array of buckets; each bucket is a linked list or tree" earns more than reciting method names.
 3. **Name the guarantee, not the vibe.** For concurrency, say *what* is guaranteed (visibility, atomicity, ordering) — that's the signal.
 4. **Admit the boundary.** "Beyond this I'd reach for the docs" beats bluffing.
-

1. Collections Internals — HashMap & ConcurrentHashMap

The concept in one line. A `HashMap` stores entries in an **array of buckets**; the key's hash decides which bucket, and each bucket holds a short list (or, when it gets long, a balanced tree) of entries that landed there.

How a HashMap is laid out

```
table (array of buckets), length is always a power of 2
index:  0      1      2      3      4      5      ...
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+
| null | | E1  | | null | | E3  | | null | | E5  |
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+
          |           |           |
          v           v           v
        [E1]->[E2]    [E3]        [E5]->[E6]->[E7]...
        (collision:   (single      (long chain -> becomes
         linked list)  entry)       a red-black TREE at >=8)
```

- **Bucket index** = `hash(key) & (table.length - 1)`. Because length is a power of two, this bitmask is a fast substitute for modulo.
- **Collisions** (two keys in the same bucket) are chained in a linked list.
- **Treeification**: since Java 8, once a bucket's chain reaches **8** entries *and* the table is at least 64 in size, that bucket converts to a **red-black tree**, so worst-case lookup in that bucket drops from $O(n)$ to $O(\log n)$.

put() — step by step

```
put("key", V):
1. h = spread(key.hashCode())           // mixes high bits down (see below)
2. i = h & (n - 1)                       // bucket index
3. bucket empty? -> place new node, done
4. else walk the chain/tree:
   - found a node whose key .equals() -> overwrite its value
   - else append a new node (treeify if chain >= 8 && table >= 64)
5. if ++size > threshold (capacity * loadFactor) -> resize()
```

The spread trick. `HashMap` doesn't use `hashCode()` raw. It XORs the high 16 bits into the low 16: `h = hc ^ (hc >>> 16)`. Why? The bucket index only looks at the *low* bits (the mask). If many keys differ only in their high bits, they'd all collide. Mixing spreads them out.

Resizing (the part interviewers love)

Default capacity **16**, load factor **0.75** → resize threshold **12**. When `size` crosses it, the table **doubles** (16 → 32 → 64 ...) and every entry is redistributed.

```
Before (n=16): entry sits in bucket i = h & 15
After (n=32): it moves to either i (stays) OR
                    i + 16 (moves up)
-> which one depends on the single new bit (h & 16)
```

Since Java 8, resizing is smart: entries in one old bucket split into just **two** new buckets (`i` and `i + oldCap`), preserving order — no full rehash of every key.

Operation	Average	Worst case (pre-Java 8)	Worst case (Java 8+)
get / put / remove	O(1)	O(n) (long chain)	O(log n) (treeified)
Iteration	O(n + capacity)	—	—

Worked example

```
Map<String, Integer> counts = new HashMap<>(); // capacity 16, threshold 12
for (String w : "a b a c a b".split(" ")) {
    counts.merge(w, 1, Integer::sum); // clean way to count
}
// counts = {a=3, b=2, c=1}
// "a","b","c" hash to some buckets; repeated keys hit the same bucket
// and overwrite via .equals() rather than adding duplicates.
```

ConcurrentHashMap — the thread-safe cousin

`HashMap` is **not** thread-safe: concurrent `put`s during a resize can corrupt the table (historically even an infinite loop). Options:

- `Collections.synchronizedMap(map)` — wraps every method in one lock. Simple, but all threads serialize on a single lock → poor throughput.
- **`ConcurrentHashMap`** — the real answer. Since Java 8 it locks **per bucket** (using a `synchronized` block on the bucket's head node + CAS for empty buckets), so threads touching different buckets don't block each other.

<code>synchronizedMap:</code>	<code>ConcurrentHashMap (Java 8+):</code>
one lock for whole map	lock is per-bucket (fine-grained)
[===== LOCK =====]	[b0][b1][b2][b3] each independently guarded
thread A waits for B	A writes b1 while B writes b3 → no contention

Key points to say: reads are **lock-free** (fields are `volatile`), writes lock only the one bucket, and there is **no locking of the whole map** — so `size()` is an estimate and iterators are **weakly consistent** (they don't throw `ConcurrentModificationException`, they reflect some state during traversal).

Traps interviewers probe

- **"What if a key's `hashCode` always returns 7?"** Every entry lands in one bucket → degrades to $O(n)$, or $O(\log n)$ after treeification. Good segue to `equals / hashCode`.
- **"Is `HashMap` ordered?"** No. Use `LinkedHashMap` (insertion/access order) or `TreeMap` (sorted by key).
- **"Can I use a mutable object as a key?"** Dangerous — if the key mutates after insertion, its hash changes and you can't find it again.
- **"`Hashtable` vs `ConcurrentHashMap`?"** `Hashtable` is legacy, locks the whole table. Prefer `ConcurrentHashMap`.

What to say (crisp version)

"A `HashMap` is an array of buckets indexed by a spread hash of the key. Collisions chain into a list that treeifies to a red-black tree past 8 entries. It resizes by doubling at 75% load. It's not thread-safe — for concurrency I use `ConcurrentHashMap`, which locks per-bucket so unrelated writes don't contend and reads stay lock-free."

2. equals() and hashCode() Contract

The concept in one line. Two objects that are `equal` **must** report the same `hashCode`, so hash-based collections (`HashMap`, `HashSet`) can find them — but two objects with the same `hashCode` don't have to be `equal`.

The contract, precisely

`Object` defines both methods, and any override must preserve these rules (from the `Object` Javadoc):

- **Consistency:** repeated calls to `equals()` return the same result, as long as the fields used in the comparison haven't changed.
- **Reflexive:** `a.equals(a)` is always `true`.
- **Symmetric:** `a.equals(b)` implies `b.equals(a)`.
- **Transitive:** `a.equals(b)` and `b.equals(c)` implies `a.equals(c)`.
- **hashCode agreement (the one interviewers care about most):** `a.equals(b) == true` → `a.hashCode() == b.hashCode()` **must** hold. The reverse is **not** required — two unequal objects are allowed to share a hash (a "hash collision"); that's normal and handled by chaining, not a bug.

```
equals() same -----> hashCode() MUST match (required)
hashCode() same ----??-----> equals() may still differ (allowed)
```

A correct implementation (Java 8+)

```
final class Point {
    private final int x;
    private final int y;

    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;           // reflexive shortcut
        if (!(o instanceof Point)) return false; // type check, see traps below
        Point p = (Point) o;
        return x == p.x && y == p.y;           // compare all fields used below
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);             // same fields as equals()
    }
}
```

`Objects.hash(x, y)` is shorthand for `Arrays.hashCode(new Object[]{x, y})` — fine for most classes. For a hot path where boxing matters, the manual Effective-Java form (`int h = 17; h = 31*h +`

`x; h = 31*h + y; )` avoids the varargs array allocation.

Records (Java 16+): `record Point(int x, int y) {}` generates `equals()`, `hashCode()`, and `toString()` for you, based on all components, following this exact contract. Mention it as the modern default — hand-rolling the boilerplate above is now mostly needed for classes that aren't records (e.g. mutable ones, or ones needing custom field selection).

What breaks if you override `equals()` but not `hashCode()`

This is the diagram to have ready. Say you override `equals()` on `Point` so that `(1,2).equals(new Point(1,2))` is `true`, but leave the default `Object.hashCode()` (identity-based) untouched.

```
HashSet<Point> seen = new HashSet<>();
seen.add(new Point(1, 2));           // object A, identity hash = 111
seen.add(new Point(1, 2));           // object B, equals A, identity hash = 222

bucket index = hash & (n-1)

      bucket(111)                bucket(222)
      +-----+                +-----+
      |  A      |                |  B      |   <- A and B are .equals() but
      +-----+                +-----+               land in DIFFERENT buckets

seen.size() == 2                  // duplicate "slipped in" -- contract broken!
seen.contains(new Point(1,2))    // looks in bucket(new hash) -- may say
                                // false even though an equal element exists
```

The lookup path (`get` / `contains`) hashes the key first to pick a bucket, **then** calls `equals()` only on entries already in that bucket. If `hashCode()` doesn't route equal objects to the same bucket, `equals()` never even gets the chance to run — so the collection behaves as if the objects were different, defeating the entire point of a value type.

Traps interviewers probe

- **"You overrode `equals` — do you need `hashCode`?"** Yes, always together. Overriding one without the other is the single most common bug in this area (IDEs and `@EqualsAndHashCode` - style annotations exist mainly to prevent it).
- **Mutable fields used in `hashCode()`** . If you put a mutable object into a `HashSet` / `HashMap` key and then change a field that `hashCode()` depends on, the object's bucket location "goes stale" — you'll never find it again by `.equals()` lookup, and `remove()` silently fails. Prefer immutable fields for hash/equals.
- **`==` vs `equals()`** . `==` compares references (or primitive values); `equals()` compares logical content. Forgetting this on wrapper types is a classic gotcha: `Integer.valueOf(127) == Integer.valueOf(127)` is `true` (cache), but `Integer.valueOf(200) == Integer.valueOf(200)` is `false` — always use `.equals()` for boxed types.
- **`instanceof` vs `getClass()` in `equals()`** . `instanceof` lets subclasses be equal to their parent type but can break symmetry if the subclass adds fields. `getClass() == o.getClass()` is stricter (exact-type match) and preferred when the class is part of an inheritance hierarchy you don't fully control. For `final` value classes it rarely matters — pick `instanceof` for simplicity, `getClass()` for safety in hierarchies.

- **Records "just work," but watch array fields.** A record with an array component gets reference-based equality for that field (arrays don't override `equals()`), so two records holding "equal" arrays still compare unequal — same trap as plain classes.

What to say (crisp version)

"The contract is one-directional: equal objects must have equal hashcodes, but equal hashcodes don't imply equal objects — collisions are expected. If you override `equals()` without `hashCode()`, a `HashMap/HashSet` can put logically-equal objects in different buckets, so lookups fail and duplicates slip in. I always generate both together from the same fields, prefer immutable fields for anything used in hashing, and reach for records in Java 16+ since they generate a contract-correct `equals / hashCode` for free."

3. JVM Memory & Garbage Collection

The concept in one line. Each thread gets a small private **stack** for method calls and local variables; every thread shares one big **heap** where actual objects live, and the **garbage collector** reclaims heap objects that nothing can reach anymore.

Stack vs Heap — who lives where

Thread A stack	Thread B stack	Heap (shared)
+-----+ frame: main() x = 5 p -----> +-----+ (primitives + refs only)	+-----+ frame: run() local = null +-----+ (one frame per method call)	+-----+ Person@0x1 name -> -----+ -----> Person@0x2 age = 30 +-----+ "Alice" (String) <--+ +-----+
Metaspace (off-heap): class metadata, method bytecode, constant pool		

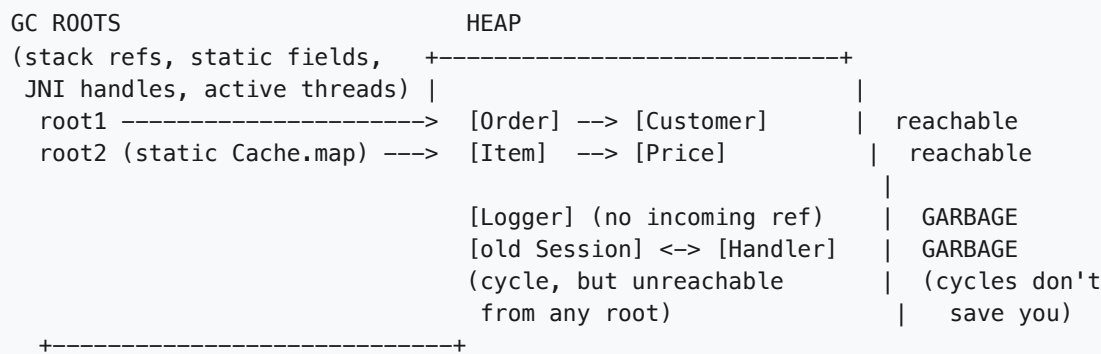
- **Stack:** one per thread, LIFO frames pushed per method call. A frame holds **primitives** (`int` , `boolean` , ...) and **references** (pointers into the heap) — never the object body itself. Popped automatically when the method returns → no GC needed for stack space.
- **Heap:** shared by all threads. Every `new` allocates the actual object here. Lives until no reference points to it anymore.
- **Metaspace** (replaced PermGen in Java 8): holds class definitions, method bytecode, the constant pool — not application objects. Grows dynamically, but a class-loader leak (e.g. reloading webapps without unloading classloaders) can still exhaust it.

Code: what goes where

```
void makeAndUse() {  
    int age = 30;           // primitive -> lives in this frame on the STACK  
    Person p = new Person("Alice", age); // 'p' (ref) -> STACK  
                                // Person object + "Alice" -> HEAP  
    process(p);             // new frame pushed; its 'p' param is a  
                                // second reference, same STACK slot pattern,  
                                // pointing at the SAME heap object  
                                // frame popped, 'age' and 'p' vanish;  
                                // object survives only if reachable elsewhere  
}
```

Two stack frames, two reference variables, but only **one** `Person` object on the heap — the stack never duplicates object data, only pointers to it.

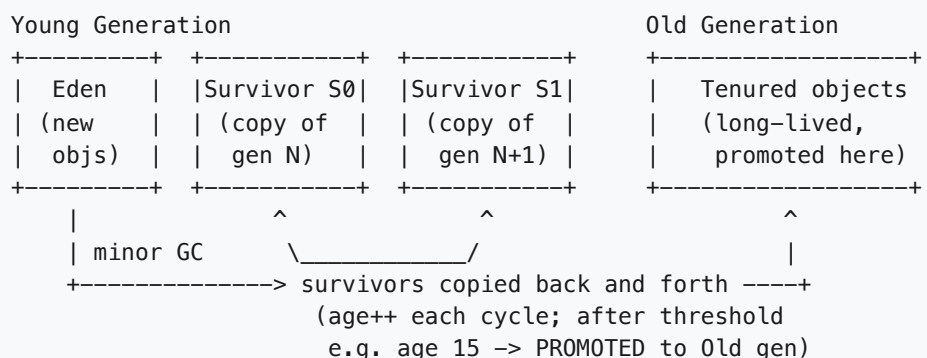
Reachability — what counts as garbage



An object is **live** only if some chain of references starting at a **GC root** reaches it. Two objects can reference each other in a cycle and still be garbage if the cycle itself is unreachable from any root — unlike naive "refcounting", the JVM's tracing collectors handle this correctly.

Generational GC — why age matters

Empirically, most objects die within milliseconds of being created (loop temporaries, boxed values, request-scoped DTOs). The heap is split so the collector can exploit that.



- **Minor GC:** collects only Eden + Survivors. Fast, frequent. Live objects copy from Eden into a Survivor space (and between the two Survivors each cycle), aging by one each time.
- **Promotion:** an object that survives enough minor GCs (or a big object that never fits in Eden) gets **tenured** into the Old generation.
- **Major / Full GC:** collects the Old generation (often the whole heap). Much slower — this is the pause that shows up as latency spikes in production.
- Because most garbage is caught cheaply in Eden, minor GCs are frequent but sub-millisecond; full GCs are rare but expensive — that trade-off is the entire point of generational design.

Collectors, one line each

Collector	One-liner
Serial	Single-threaded, stop-the-world. Fine for small heaps / single-core boxes.

Collector	One-liner
Parallel (throughput)	Multi-threaded stop-the-world; maximizes throughput, ignores pause time.
G1 (Garbage First)	Splits heap into many regions, collects the "garbage-first" ones; balances throughput and pause time. Default since Java 9.
ZGC	Concurrent, region-based, sub-millisecond pauses even on multi-terabyte heaps — trades some throughput/CPU for latency.

Traps interviewers probe

- **"Can Java leak memory even with GC?"** Yes — GC only frees *unreachable* objects. Classic leaks: entries piling up in a `static Map / List` that's never cleared, listeners or callbacks registered but never unregistered, unclosed `InputStream / Connection` resources (use `try-with-resources`), and `ThreadLocal` values never removed in pooled threads.
- **"Does `finalize()` guarantee cleanup?"** No — it's unreliable (no timing guarantee, may never run) and **deprecated for removal**. Use `try-with-resources / Closeable` or `Cleaner` instead.
- **"Does calling `System.gc()` force a collection?"** No — it's only a *hint* to the JVM; the collector is free to ignore it entirely.
- **`OutOfMemoryError` vs `StackOverflowError` ?** `OutOfMemoryError`: Java heap space means the **heap** is full of reachable (or too-large) objects and GC can't free enough. `StackOverflowError` means one thread's **stack** exhausted its frames — almost always unbounded/infinite recursion, unrelated to heap size.
- **"Are Eden/Survivor/Old the same for every collector?"** The generational Young/Old split is conceptually universal (except ZGC, which is mostly region-based without strict generations pre-JDK 21, and gains generational mode in later releases) — good nuance to mention if pushed.

What to say (crisp version)

"Each thread has a stack for frames holding primitives and references; the heap is shared and holds the actual objects, reachable from GC roots like stack refs and statics. GC is generational: new objects go into Eden, survive minor GCs by copying between Survivor spaces, age, and get promoted to Old gen — because most objects die young, this keeps the expensive full GCs rare. G1 is the default collector today, ZGC if you need sub-millisecond pauses. GC doesn't prevent leaks — a forgotten static collection or unregistered listener keeps things reachable forever."

4. Generics & Type Erasure

The concept in one line. Generics let the **compiler** catch type mistakes and remove casts, but at **runtime** the type parameter is gone — `List<String>` and `List<Integer>` are both just `List` inside the JVM.

Why generics exist

```
List raw = new ArrayList();
raw.add("hello");
raw.add(42);                      // compiles fine, no warning pre-generics
String s = (String) raw.get(1);    // boom: ClassCastException at runtime

List<String> typed = new ArrayList<>();
typed.add("hello");
typed.add(42);                    // COMPILE ERROR – caught immediately
String s2 = typed.get(0);         // no cast needed, compiler inserts it
```

- **Raw List** : any object goes in, mistakes surface late (at the cast, far from the bug).
- **List<String>** : the compiler enforces the contract *and* auto-inserts the cast on `get` . Same bytecode underneath — the win is purely at compile time.

Type erasure — the centerpiece

Generics are a **compile-time-only** feature. The compiler checks your types, then strips them out and inserts casts, producing plain old bytecode that predates generics (Java 5).

SOURCE (compile time)	BYTECODE (runtime, after erasure)
<code>List<String> l = new ArrayList<>();</code>	<code>List l = new ArrayList(); // raw</code>
<code>l.add("hi");</code>	<code>l.add("hi"); // unchanged</code>
<code>String s = l.get(0);</code>	<code>String s = (String) l.get(0); // CAST inserted</code> <code>for T with no bound -> erased to Object</code> <code>for <T extends Comparable<T>> -> erased to Comparable</code>

At runtime there is exactly **one** class file for `ArrayList` , not one per type argument.

`List<String>.class == List<Integer>.class` — literally the same `Class` object.

```
List<String> ls = new ArrayList<>();
List<Integer> li = new ArrayList<>();
System.out.println(ls.getClass() == li.getClass()); // true – both ArrayList.class
```

Consequences of erasure (the "gotchas")

Can't do	Why
new T[10]	T is erased to Object; JVM can't allocate the right array type at runtime
x instanceof List<String>	only the raw List shape exists at runtime; "instanceof List<?>" is the legal form
two overloads differing only in generic type	void f(List<String> a) void f(List<Integer> a) <- same erased signature f(List) -> DUPLICATE METHOD error
static field of type T	class Box<T> { static T value; } <- illegal; T doesn't exist per-instance, and there's only ONE Box.class shared by all Box<X>

```
class Box<T> {  
    // T[] arr = new T[10];          // compile error  
    T[] arr = (T[]) new Object[10]; // the usual workaround: unchecked cast  
}
```

Bounded types

`<T extends Comparable<T>>` restricts `T` to types that can compare themselves — lets you call `compareTo` inside a generic method without casting to `Object`.

```
static <T extends Comparable<T>> T max(List<T> list) {  
    T best = list.get(0);  
    for (T item : list) {  
        if (item.compareTo(best) > 0) best = item;  
    }  
    return best;  
}  
max(List.of(3, 7, 2));          // T = Integer, works: Integer implements Comparable<Integer>
```

Wildcards — PECS

Wildcards (?) let a generic type vary *at the call site* instead of the declaration. The mnemonic every interviewer wants to hear:

PECS: Producer Extends, Consumer Super

List<? extends Number> you only READ from it (produces Number/subtypes)
 -> add(x) is FORBIDDEN (compiler can't know the real element type)
 -> Number n = list.get(0) OK

List<? super Integer> you only WRITE Integers into it (consumes them)
 -> list.add(5) OK
 -> Object o = list.get(0) that's all you get back — type is unknown above Integer

```
// copy: read from src (producer), write into dest (consumer)
static void copy(List<? extends Number> src, List<? super Number> dest) {
    for (Number n : src) {
        dest.add(n);          // dest.add(src.get(i)) also works, one line
    }
    // src.add(1);             // compile error – src might really be List<Double>
    // Number n = dest.get(0); // compile error – dest might really be List<Object>
}

List<Integer> ints = List.of(1, 2, 3);
List<Object> sink = new ArrayList<>();
copy(ints, sink);          // Integer "extends" Number, Object "super" Number
```

If you need to both read *and* write with real type safety, use an exact type (`List<Number>`), not a wildcard — wildcards exist precisely to widen what a caller can pass in, at the cost of restricting what you can do with it inside.

Arrays vs generics: covariant vs invariant

Arrays are COVARIANT: <pre>Object[] a = new String[3]; a[0] = 42; // compiles, // throws // ArrayStoreException // at RUNTIME</pre>	Generics are INVARIANT: <pre>List<Object> l = new ArrayList<String>(); // COMPILER ERROR – not allowed, period</pre>
---	---

Arrays remember their real element type at runtime and check every store (`ArrayStoreException` if you violate it). Generics have no runtime type to check against — so the compiler simply refuses the invariant-breaking assignment up front. This mismatch is exactly why you can't safely do `new T[]` — the runtime array-store check that arrays rely on has nothing to check.

Traps interviewers probe

- **"Heap pollution / unchecked warnings — what causes them?"** Mixing raw types with generic types, or casting where erasure hides the real type, e.g. `List<String>[] arr = (List<String>[]) new List[10];` compiles with an *unchecked* warning because the JVM can't verify every element really holds `List<String>` at runtime.
- **"Why can't I overload `foo(List<String>)` and `foo(List<Integer>)`?"** Same erased signature `foo(List)` — the compiler sees a duplicate method, not two overloads.
- **"What's a bridge method?"** When a generic method is overridden with a more specific type (e.g. `Comparable<T>.compareTo(T)` overridden as `compareTo(MyClass)`), the compiler generates a synthetic **bridge method** with the erased signature (`compareTo(Object)`) that casts and delegates — so overriding/polymorphism still works correctly after erasure. You rarely write these; you just need to know why `getDeclaredMethods()` sometimes shows an extra method you never wrote.
- **"Is `List<Object>` the same as raw `List`?"** No — `List<Object>` still enforces "only `Object`-compatible types, checked at compile time"; raw `List` disables checking entirely and generates unchecked warnings.

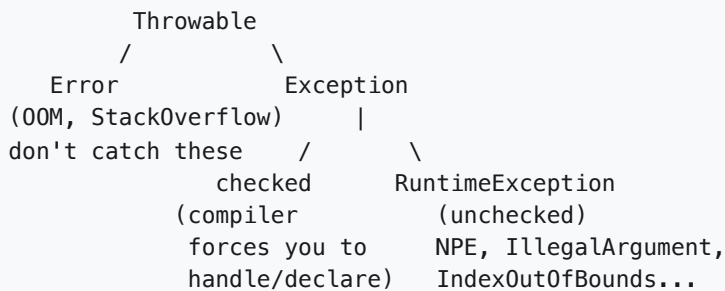
What to say (crisp version)

"Generics are compile-time-only — erasure strips the type parameters and inserts casts, so `List<String>` and `List<Integer>` share one `List.class` at runtime. That's why you can't do `new T[]`, can't `instanceof List<String>`, and can't overload on generic type alone. For APIs I use bounded types like `<T extends Comparable<T>>` and wildcards with PECS — producer extends when I only read, consumer super when I only write — to keep callers flexible without giving up compile-time safety."

5. Exceptions, try-with-resources & Optional

The concept in one line. Exceptions are Java's way of signaling "something went wrong and the normal return value can't answer the question" — `try-with-resources` guarantees cleanup, and `Optional` is a typed container that forces the caller to *think* about absence instead of silently NPE-ing.

The exception hierarchy



- **Error** — JVM-level failures (`OutOfMemoryError`, `StackOverflowError`). You don't catch or recover from these; the process is usually in a bad state.
- **Checked exceptions** (`IOException`, `SQLException`) — extend `Exception` directly. The compiler forces you to either `catch` or declare `throws`. Meant for *recoverable* conditions the caller can reasonably act on (retry, fallback).
- **Unchecked / RuntimeException** (`NullPointerException`, `IllegalArgumentException`, `IllegalStateException`) — no compiler enforcement. Meant for **programming errors** — bugs that should surface loudly, not be caught everywhere defensively.

When to use which. Rule of thumb: if the caller can do something useful with the failure (retry a network call, ask the user for different input) → checked. If it's a contract violation that indicates a bug (passed `null` where forbidden, called a method in the wrong state) → unchecked. Most modern Java APIs (and most interviewers) lean toward **unchecked by default** — checked exceptions tend to leak into every signature up the call stack and get swallowed by lazy `catch (Exception e) {}` blocks.

try / catch / finally semantics

```
try { risky() }           -> runs
catch (X e) { handle() } -> runs only if risky() throws an X
finally { cleanup() }     -> ALWAYS runs: normal exit, exception, or return
```

`finally` runs even if the `try` or `catch` block returns, breaks, or continues. The one trap interviewers love:


```
static int trap() {
    try {
        return 1;
    } finally {
        return 2;          // <-- this WINS. Method returns 2, silently.
    }
}
```

A `return` (or `throw`) inside `finally` **discards** whatever the `try / catch` was about to return or throw. This is almost always a bug — never put a `return` in `finally`.

try-with-resources (AutoCloseable)

The old, leak-prone way:

```
FileInputStream in = new FileInputStream("a.txt");
try {
    use(in);
} finally {
    in.close();          // if use() AND close() both throw, one exception is lost;
                        // if you forget this finally entirely, you leak a file handle
}
```

The clean way (Java 7+):

```
try (FileInputStream in = new FileInputStream("a.txt");
    FileOutputStream out = new FileOutputStream("b.txt")) {
    transfer(in, out);
} // both resources' close() called automatically – no finally needed
```

Any class implementing `AutoCloseable` (`close()` throws `Exception`) or `Closeable` (`close()` throws `IOException`) can be used here — streams, `Connection`, `Scanner`, locks.

Close ordering — reverse of declaration:

```
try (A a = ...; B b = ...; C c = ...) { body }
```

opened: A → B → C
 closed: C → B → A (reverse — last opened, first closed)

This mirrors stack unwinding: the resource that depends on the others (opened last) is torn down first.

Suppressed exceptions. If `body` throws *and* `close()` also throws, the `body`'s exception is the one propagated — the `close()` exception isn't lost, it's attached as a **suppressed exception**, retrievable via `getSuppressed()`:

```
try (Resource r = new Resource()) {
    throw new RuntimeException("body failed");
} // if r.close() also throws, that exception is suppressed, not swallowed
// caller sees "body failed", with r's close-exception in ex.getSuppressed()
```

This is strictly better than the manual `finally` version, where the `close()` exception used to silently **replace** the original one, hiding the real cause.

Best practices

- **Don't swallow exceptions.** `catch (Exception e) {}` (empty body) hides bugs — at minimum log it, or don't catch it at all.
- **Catch specific types, not `Exception` or `Throwable`.** Catching `Exception` broadly can accidentally swallow bugs (`NullPointerException`) alongside the real recoverable case you meant to handle. Catching `Throwable` risks intercepting `Errors` you can't safely recover from.
- **Wrap-and-rethrow with `cause`,** don't lose the original stack trace:

```
try {
    repository.save(order);
} catch (SQLException e) {
    throw new OrderPersistenceException("failed to save order " + order.getId(), e); // cause
}
```

Always pass the original exception as the **`cause`** (`e`) — dropping it destroys the stack trace that would've told you *why* it really failed.

- **Custom exceptions** should extend `RuntimeException` (usually) and add context fields, not just a message string:

```
class OrderPersistenceException extends RuntimeException {
    OrderPersistenceException(String msg, Throwable cause) { super(msg, cause); }
}
```

Optional — expressing absence without null

Purpose: `Optional<T>` is a container that is either present (holds a `T`) or empty. It makes "this might not have a value" visible **in the method signature**, instead of a `null` that every caller has to remember to check.

<code>Optional<T>:</code>	
<code>+-----+</code>	<code>+-----+</code>
<code> value = X </code>	<code> (empty) </code>
<code>+-----+</code>	<code>+-----+</code>
<code>.isPresent()->true</code>	<code>.isPresent()->false</code>
<code>.get() -> X</code>	<code>.get() -> throws NoSuchElementException</code>

Correct use — chain, don't unwrap-and-check:

```
Optional<User> user = repository.findById(id);

String displayName = user
    .map(User::getName)           // transform if present, stays empty if not
    .filter(name -> !name.isBlank())
    .orElseGet(() -> "Anonymous"); // supplier only runs if empty
```

Misuse #1 — `Optional` as a field or constructor parameter.

```
class User {
    private Optional<String> nickname;    // BAD: Optional isn't Serializable and adds
                                        // an indirection layer for no benefit – just
                                        // let the field be null internally, expose
                                        // Optional only from the *getter* if at all.
}
```

`Optional` was designed as a **return type** to signal absence to a caller — not as a general-purpose "nullable" wrapper for fields, parameters, or collection elements.

Misuse #2 — Optional as a method parameter.

```
void createUser(String name, Optional<String> nickname) { ... }    // BAD
void createUser(String name, String nickname /* nullable, or overload */) { ... } // prefer t
```

Forces every caller to wrap a value in `Optional.of(...)` just to call the method — overloading or accepting a nullable value is simpler.

Misuse #3 — calling `get()` without checking.

```
Optional<User> user = repository.findById(id);
User u = user.get();           // BAD: throws NoSuchElementException if empty –
                               // exactly the NPE-shaped bug Optional was meant to prevent!
```

`orElse` VS `orElseGet` — eager vs lazy

```
orElse(expensiveDefault())    -> expensiveDefault() ALWAYS runs, even if present
orElseGet(() -> expensive())  -> supplier runs ONLY if empty (lazy)
```

```
// BAD: buildDefaultUser() runs on every call, even when 'found' is present
User u = found.orElse(buildDefaultUser());

// GOOD: buildDefaultUser() only runs when found is actually empty
User u = found.orElseGet(this::buildDefaultUser);
```

`orElse` takes an already-computed value; `orElseGet` takes a `Supplier` evaluated lazily. If the default is a constant (`orElse("N/A")`) it doesn't matter — if it's a method call with side effects or cost, `orElseGet` is the correct choice.

Traps interviewers probe

- **"What's wrong with `catch (Throwable t)`?"** It also catches `Errors` like `OutOfMemoryError` and `StackOverflowError` — states you generally can't safely recover from.
- **"Should you use exceptions for control flow?"** No — throwing/catching is expensive (stack trace capture) and it obscures normal logic paths. Use return values, `Optional`, or loop constructs instead of throwing to "break out" of normal flow.
- **"What does `Optional.get()` throw, and when?"** `NoSuchElementException`, if the `Optional` is empty. It's the same footgun as dereferencing `null` — always guard with `isPresent()`, or better, use `map` / `orElseGet` / `orElseThrow` so the empty path is explicit.

- **"orElse vs orElseGet ?"** Eager vs lazy — `orElse` 's argument is evaluated regardless of presence; `orElseGet` 's supplier only runs on empty. Matters for cost/side effects.
- **"Return in finally?"** Silently discards the `try / catch` result — treat it as a bug smell.

What to say (crisp version)

"Errors are unrecoverable JVM failures — don't catch them. Checked exceptions force the caller to handle a recoverable condition; unchecked ones signal programming bugs and don't need to be declared. `finally` always runs, even across a return, which is why a return inside finally is a trap — it silently overrides the try's result. For resource cleanup I use `try-with-resources`: it closes resources in reverse order automatically and attaches a close-time exception as suppressed rather than losing it. For exceptions I catch specific types, never swallow them, and wrap-and-rethrow with the original cause preserved. `Optional` is for return types that might be absent — I chain it with `map/filter/orElseGet`, avoid `Optional` fields or parameters, and never call `get()` without checking, since that's just `NPE` with extra steps. And `orElseGet` is lazy, `orElse` is eager — I use `orElseGet` whenever the default has real cost."

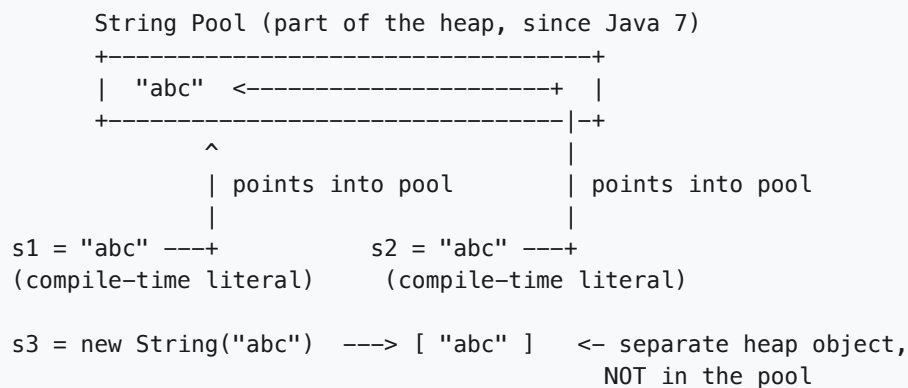
6. Immutability & the String Pool

The concept in one line. `String` is immutable — once created, its character data never changes — and the JVM exploits that guarantee to safely **share** identical literals in a pool instead of allocating a new object every time.

Why String is immutable

- **Security.** Strings are used for class names, file paths, network hosts, DB URLs. If a `String` could be mutated after a security check ran, an attacker could swap the value between the check and the use (a "check-then-act" race). Immutability closes that hole.
- **Thread-safety for free.** No internal state ever changes, so any number of threads can read the same `String` with no locking, no visibility bugs, no defensive copying.
- **HashCode caching.** `String.hashCode()` computes once and caches the result in a private `hash` field, because the value can never change. Huge win when Strings are used as `HashMap` keys (see topic 1) — the hash is computed once, not on every `put / get`.
- **String pool sharing.** Because two equal Strings are guaranteed to stay equal forever, the JVM can let many references point at **one** shared object. If Strings were mutable, changing one literal's value would corrupt every other variable pointing at the "same" pooled string.

The string pool — the classic gotcha, diagrammed



```
String s1 = "abc";
String s2 = "abc";
String s3 = new String("abc");

s1 == s2      // true  -> both refer to the SAME pooled object
s1 == s3      // false -> s3 is a distinct object on the heap
s1.equals(s3) // true  -> equals() compares char content, not identity

String s4 = s3.intern(); // looks up/adds "abc" in the pool, returns pooled ref
s1 == s4      // true
```

- Any **compile-time constant** string literal is automatically interned and placed in the pool the first time the class is loaded.
- `new String("abc")` explicitly forces creation of a fresh object on the heap that is a *copy* of the pooled data — it deliberately opts out of sharing.
- `==` compares **references** (object identity); `.equals()` compares **content**. For Strings you almost always want `.equals()` — `==` "works" only by the accident of pool sharing and breaks the moment a `new String(...)` or runtime concatenation is involved.
- Runtime-built strings (`"a" + someVariable`) are **not** interned automatically — only literals and compile-time constant expressions (`"a" + "b"`, both literals) are.

Concatenation in a loop → $O(n^2)$, fix with `StringBuilder`

```
// BAD: each += creates a brand-new String, copying everything so far
String s = "";
for (int i = 0; i < n; i++) {
    s += i;           // new object, length grows → total work 0(1+2+...+n) = 0(n^2)
}
```

```
iteration 1: copy 1 char  → new String
iteration 2: copy 2 chars → new String  (previous one discarded, becomes garbage)
iteration 3: copy 3 chars → new String
...
iteration n: copy n chars → new String
total copying work: 1+2+...+n = 0(n^2)
```

```
// GOOD: StringBuilder holds one mutable char[] buffer, amortized 0(1) append
StringBuilder sb = new StringBuilder();
for (int i = 0; i < n; i++) {
    sb.append(i);    // grows internal buffer only when full (doubling), 0(n) total
}
String s = sb.toString(); // one final copy into an immutable String
```

The compiler already does this for you for a **single fixed** concatenation expression (`"a" + b + "c"` compiles to one `StringBuilder` chain), but it **cannot** do it across loop iterations — each `+=` in a loop is a separate statement, so you must use `StringBuilder` explicitly.

Writing your own immutable class

Recipe: `final` class (no subclass can add mutable behavior), `final` fields set only in the constructor, no setters, and **defensive copies** anywhere a mutable object crosses the boundary — on the way in (constructor) and on the way out (getter).

```

public final class Order {                                // 1. final class
    private final String id;                               // 2. final fields
    private final List<String> items;                      // reference is final,
                                                         // but List itself is mutable!

    public Order(String id, List<String> items) {
        this.id = id;
        this.items = new ArrayList<>(items);              // 3. defensive copy IN
    }

    public String getId() { return id; }

    public List<String> getItems() {
        return Collections.unmodifiableList(items); // 4. defensive copy/wrap OUT
        // return new ArrayList<>(items);              // (alternative: real copy)
    }
    // 5. no setters at all
}

```

caller's list	Order's private copy	what getItems() returns
[A, B, C] --copy in-->	[A, B, C] --wrap-->	unmodifiable view of [A, B, C]
caller mutates original	unaffected	caller cannot mutate this either

Without step 3, the caller could mutate the `List` they passed in *after* construction and silently change the "immutable" `Order`. Without step 4, the caller could grab the internal `List` via the getter and mutate it directly — same bug, different door.

Benefits of immutability

- **Inherently thread-safe** — shareable across threads with zero synchronization, since there's no mutable state to race on.
- **Safe as `HashMap` / `HashSet` keys** — the hash code can't change after insertion, so the entry never becomes "lost" in the wrong bucket (see topic 1's mutable-key trap).
- **Cacheable / shareable** — safe to hand out the same instance everywhere (string pool, `Integer` cache for -128..127, etc.) since no one can corrupt a shared copy for others.
- **Easy to reason about** — a method that takes an immutable object can't have its input changed underneath it; no defensive copying needed by the *caller*.

Traps interviewers probe

- **"Is `s1 == s2` true for two `"abc"` literals?"** Yes — both point to the same pooled object. Change one to `new String("abc")` and it flips to `false`. This is the #1 trick question; always answer with `==` = identity, `.equals()` = content, and pool sharing only applies to literals/interned strings.
- **"Does `substring()` leak memory?"** Historically (pre-Java 7) `substring()` shared the parent's backing `char[]`, so a tiny `substring` of a huge string kept the whole array alive — a real memory leak. Since **Java 7u6**, `substring()` copies only the needed characters into a new array, so this specific leak no longer exists. Worth mentioning as a "used to be true" trap.
- **"Should I call `intern()` on every string?"** No — overusing `intern()` fills the pool (which has its own memory cost and, pre-Java 8, lived in limited PermGen) and the lookup itself isn't free.

Only intern strings you know are long-lived, high-duplication, and compared with `==` for performance reasons.

- **"final makes an object immutable" — true or false? False.** `final` only prevents the *reference* from being reassigned; it says nothing about the referenced object's internal state. `final List<String> items` still lets you call `items.add(...)`. Immutability is a property of the *class design* (no mutators, defensive copies), not of the `final` keyword.
- **"Can a supposedly-immutable object still be mutated?"** Yes, via **escaping mutable state** — if a getter returns the internal mutable field directly (no copy/wrap), the caller can mutate it and the change is reflected inside the "immutable" object. This is exactly what defensive copying on the way out (step 4 above) prevents.

What to say (crisp version)

"String is immutable so the JVM can safely intern and share literals in a string pool, cache the hash code, and guarantee thread-safety with no locking. Literals with the same content share one pooled reference, so `==` is true between them, but `new String(...)` always creates a separate heap object, so `==` is false even though `.equals()` is true — that's the classic gotcha. Building strings in a loop with `+=` is $O(n^2)$ because each concatenation copies everything so far; `StringBuilder` fixes that with one mutable buffer. To write my own immutable class I make it `final`, make fields `final`, give it no setters, and defensively copy any mutable input or output — otherwise a caller can mutate state through the constructor argument or the getter and 'final' alone won't stop them, since `final` only locks the reference, not the object it points to."

7. Lambdas, Streams & CompletableFuture

The concept in one line. A lambda is just an object implementing a one-method interface; a stream is a **lazy pipeline** over data that does nothing until a terminal op pulls values through it; `CompletableFuture` is that same pipeline idea applied to **async** results instead of collections.

Lambdas & functional interfaces

A lambda needs a target type — an interface with exactly **one** abstract method (a "functional interface", marked `@FunctionalInterface`). The lambda body becomes that method's implementation.

```
Function<T,R>   T -> R       apply(t)
Predicate<T>    T -> boolean test(t)
Supplier<T>     () -> T       get()
Consumer<T>     T -> void     accept(t)
```

```
Function<String, Integer> len = s -> s.length();
Predicate<Integer> isEven  = n -> n % 2 == 0;
Supplier<List<String>> newList = ArrayList::new;
Consumer<String> print = System.out::println;
```

Method references (`Class::method`) are just shorthand for a lambda that calls an existing method — `String::length` instead of `s -> s.length()`. Four flavors: static (`Integer::parseInt`), instance-on-particular-object (`list::add`), instance-on-first-arg (`String::length`), and constructor (`ArrayList::new`).

Streams — lazy until the terminal op

A stream pipeline has three parts: a **source**, zero or more **intermediate** operations (`filter`, `map`, `sorted` — all lazy, they just describe work), and one **terminal** operation (`collect`, `reduce`, `forEach`, `count` — eager, it actually runs everything).

```
source -> filter -> map -> collect
  |         |         |         |
  (list)   (describe) (describe) (RUN: pulls elements
                                one at a time through
                                filter+map, element-by-
                                element, not stage-by-stage)
```

Nothing executes until `collect()` (or another terminal op) is called — intermediate ops just build a recipe. Also note the *pull* direction: each element is drawn through the whole chain before the next element starts (not "run filter over everyone, then map over everyone").

```
List<String> names = List.of("Ann", "Bob", "Ale", "Sam", "Amy");

Map<Character, Long> countByFirstLetter = names.stream()
    .filter(n -> n.length() <= 3)           // lazy: just noted
    .map(String::toUpperCase)               // lazy: just noted
    .collect(Collectors.groupingBy(
        n -> n.charAt(0),
        Collectors.counting()));           // terminal: pipeline actually runs
// {A=3, B=1, S=1} (from ANN, ALE, AMY, BOB, SAM)
```

If you never call a terminal operation, the `filter` / `map` lambdas are **never invoked at all** — easy to demonstrate by putting a `System.out.println` inside `map` and calling only `Stream.of(...).map(...)` with no terminal op: nothing prints.

Common collectors

```
List<String> upper = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());

Map<String, Integer> byName = names.stream()
    .collect(Collectors.toMap(n -> n, String::length)); // key -> value

Map<Boolean, List<String>> shortVsLong = names.stream()
    .collect(Collectors.partitioningBy(n -> n.length() <= 3)); // true/false buckets

Map<Integer, List<String>> byLength = names.stream()
    .collect(Collectors.groupingBy(String::length)); // arbitrary-key buckets

String csv = names.stream()
    .collect(Collectors.joining(", ", "[", "]")); // "[Ann, Bob, ...]"
```

`groupingBy` vs `partitioningBy`: `partitioning` always produces exactly **two** buckets (`true` / `false`) even if one is empty; `grouping` produces one bucket per distinct key.

parallelStream — helps sometimes, hurts often

```
sequential: [----- one thread, in order -----]
parallel:   [-t1-][-t2-][-t3-][-t4-] (fork/join pool,
                                     merge results,
                                     extra overhead)
```

`parallelStream()` splits the source and processes chunks on the **common ForkJoinPool**. It helps when: the dataset is large, the per-element work is CPU-heavy, and the source splits cheaply (arrays, `ArrayList` — not `LinkedList` or I/O-backed sources).

It hurts when: - **The data is small** — splitting/merging overhead outweighs the work. - **There's shared mutable state** — e.g. accumulating into a plain `ArrayList` from multiple threads inside `forEach` is a race; use a proper collector (thread-safe merge) instead. - **It shares the common pool with everything else** — one slow parallel stream can starve unrelated parallel work (and `CompletableFuture` defaults use the same pool — see below).

```
// BUG: shared mutable ArrayList, unsynchronized concurrent adds
List<Integer> results = new ArrayList<>();
list.parallelStream().forEach(x -> results.add(x * 2)); // data race, can lose elements

// FIX: let the collector do the merging safely
List<Integer> results = list.parallelStream()
    .map(x -> x * 2)
    .collect(Collectors.toList());
```

CompletableFuture — async composition

`CompletableFuture<T>` represents a value that will exist later. You chain work onto it the same way you chain stream operations — each step describes what to do *when the previous stage completes*.

```
supplyAsync ----> thenApply ----> thenCompose(other async)
    |                               |
    (fetch A)                     (fetch B using A's result)

thenCombine:
  futureA ----\
                >---- combine(a, b) ----> result
  futureB ----/
```

- **thenApply** — transform the result synchronously ($T \rightarrow U$), like `map`.
- **thenCompose** — the result *is* another future; flattens `Future<Future<U>>` into `Future<U>`, like `flatMap`. Use when step 2 depends on step 1's value and is itself async.
- **thenCombine** — join two *independent* futures once both finish.
- **exceptionally** — recover from a failure earlier in the chain, like `catch`.

```
CompletableFuture<User> userFuture =
    CompletableFuture.supplyAsync(() -> fetchUser(id));

CompletableFuture<Order> orderFuture = userFuture
    .thenCompose(user -> CompletableFuture.supplyAsync(() -> fetchOrder(user.id())));

CompletableFuture<String> summary = userFuture
    .thenCombine(orderFuture, (user, order) -> user.name() + " ordered " + order.item())
    .exceptionally(ex -> "Could not build summary: " + ex.getMessage());

summary.join(); // block only at the very end, e.g. in a test or main()
```

Traps interviewers probe

- **"What happens if you reuse a stream?"** A stream can be consumed by its terminal op exactly **once** — calling another terminal op on the same stream throws `IllegalStateException: stream has already been operated upon or closed`.
- **"Is it OK to mutate state inside `map`?"** `map` / `filter` are meant to be side-effect-free; mutating shared state inside them breaks under `parallelStream` and makes the pipeline's behavior depend on execution order/laziness assumptions you can't rely on. Use `forEach` (still risky for shared mutable state) or, better, a collector.

- **"What does `Collectors.toMap` do on a duplicate key?"** Throws `IllegalStateException: Duplicate key` — pass a merge function (`toMap(k, v, (a, b) -> b)`) to resolve collisions instead of crashing.
- **"Is `parallelStream` always faster?"** No — for small collections or I/O-bound work the fork/join overhead and shared-pool contention usually make it *slower*; benchmark, don't assume.
- **"What's the danger of blocking inside a `CompletableFuture` callback?"** By default `thenApply` / `thenCompose` /etc. run on the shared common `ForkJoinPool`. A blocking call (JDBC, sleep, synchronous HTTP) inside one of those callbacks can starve that pool for the whole JVM. Use the `*Async` variants with an explicit `Executor` for blocking work.

What to say (crisp version)

"Streams are lazy — intermediate ops like map and filter just build a pipeline description, and nothing runs until a terminal op like collect or reduce pulls elements through it, one element at a time. Collectors like groupingBy, partitioningBy, and toMap give me the common list-to-map/bucket reshaping, though toMap throws on duplicate keys unless I supply a merge function. parallelStream can help on large, CPU-bound, easily-splittable data, but it's dangerous with shared mutable state and shares the same ForkJoinPool that CompletableFuture uses by default — so I avoid blocking calls in either without an explicit executor. CompletableFuture lets me compose async work the same way: thenApply for a sync transform, thenCompose when the next step is itself async and depends on the first result, and thenCombine to join two independent futures."

8. Threads — Creation & Lifecycle

The concept in one line. A `Thread` is an OS-backed unit of execution that moves through a well-defined set of **states** (NEW → RUNNABLE → maybe BLOCKED/WAITING → TERMINATED), and Java gives you three ways to hand it a task — extend `Thread`, implement `Runnable`, or implement `Callable` when you need a result back.

Three ways to create a thread

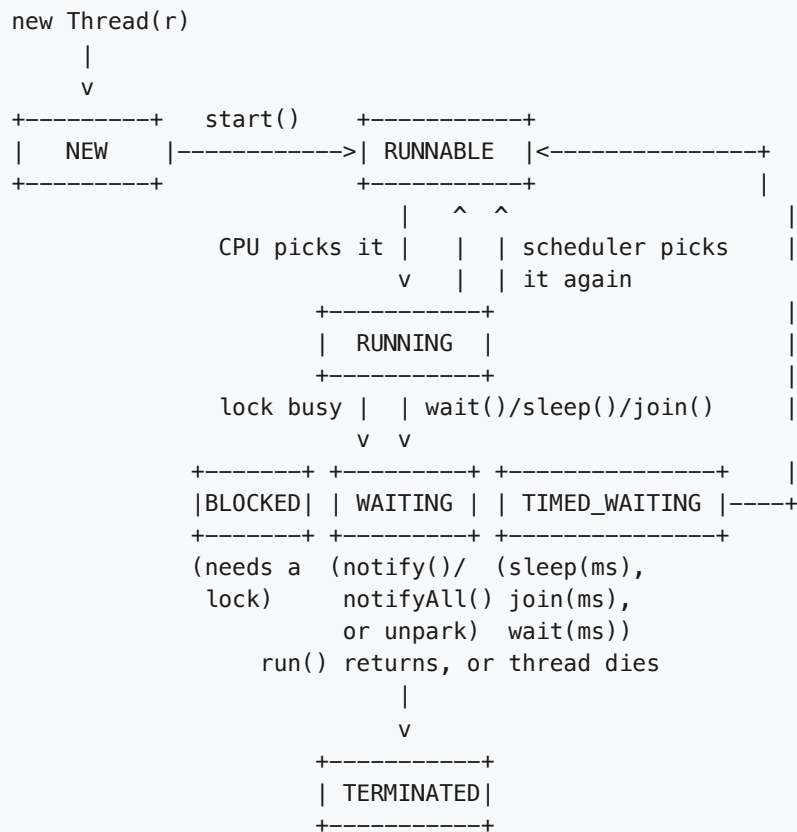
```
// 1) Extend Thread – you inherit all of Thread's baggage just to override run()
class WorkerThread extends Thread {
    @Override public void run() { System.out.println("working"); }
}
new WorkerThread().start();

// 2) Implement Runnable – preferred: decouples "the task" from "the thread"
Runnable task = () -> System.out.println("working");
new Thread(task).start();

// 3) Callable + Future – when the task needs to RETURN a value or throw a checked exception
Callable<Integer> job = () -> 40 + 2;
ExecutorService pool = Executors.newSingleThreadExecutor();
Future<Integer> future = pool.submit(job);
Integer result = future.get(); // blocks until the callable finishes
pool.shutdown();
```

Why prefer Runnable/Callable over extending Thread: - Java has single inheritance — extending `Thread` burns your one `extends` slot. - `Runnable` / `Callable` separate "what to do" from "how it runs," so the same task can be handed to a plain `Thread`, an `ExecutorService`, or scheduled — no rewrite. - `Callable<V>` can return a value (`V`) and throw checked exceptions; `run()` cannot do either.

Thread lifecycle — state diagram



- **NEW** — object created, `start()` not yet called.
- **RUNNABLE** — eligible to run; the OS scheduler decides *when* it actually gets CPU time (Java doesn't distinguish "ready" from "running" as separate `Thread.State` values — both are `RUNNABLE`).
- **BLOCKED** — waiting to acquire a `synchronized` lock held by another thread.
- **WAITING** — parked indefinitely by `Object.wait()`, `Thread.join()`, or `LockSupport.park()` until another thread calls `notify()` / `notifyAll()` / `unpark()`, or the joined thread finishes.
- **TIMED_WAITING** — same as `WAITING` but with a timeout: `sleep(ms)`, `wait(ms)`, `join(ms)`.
- **TERMINATED** — `run()` returned or threw; the thread is done and **cannot be restarted**.

start() vs run() — the classic trap

```
Thread t = new Thread(() -> System.out.println(Thread.currentThread().getName()));

t.run();    // prints "main" -> ran INLINE on the calling thread, no new thread!
t.start();  // prints "Thread-0" -> actually spawns a new call stack / OS thread
```

`run()` is just a plain method. Calling it directly executes the task **synchronously on the current thread** — no concurrency happens, and no exception is thrown, which makes the bug easy to miss. `start()` is the only thing that asks the JVM/OS to allocate a new thread of execution, which then invokes `run()` on that new thread.

t.run() called directly:
main thread ---run()----> done
(still just "main")

t.start() called:
main thread ---start()----> returns immediately
new thread -----> run() executes concurrently

join(), sleep(), interrupt()

```
Thread worker = new Thread(() -> {
    for (int i = 0; i < 5; i++) {
        if (Thread.currentThread().isInterrupted()) {
            System.out.println("interrupted, cleaning up");
            return; // exit gracefully
        }
        try {
            Thread.sleep(200); // throws InterruptedException
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt(); // RESTORE the flag!
            return;
        }
    }
});

worker.start();
Thread.sleep(300);
worker.interrupt(); // politely ASKS worker to stop; doesn't force-kill it
worker.join();      // main blocks here until worker terminates
System.out.println("worker finished");
```

- **join()** — blocks the *calling* thread until the target thread terminates (or timeout elapses). Used to wait for a worker's result before continuing.
- **sleep(ms)** — pauses the current thread, does **not** release any locks it holds.
- **interrupt()** — sets an internal boolean flag on the target thread; it does **not** stop the thread by itself. Cooperative cancellation only works if the target checks `isInterrupted()` or is blocked in a method that reacts to it (`sleep`, `wait`, `join`) by throwing `InterruptedException`.
- **Golden rule:** if you catch `InterruptedException` and can't rethrow it, call `Thread.currentThread().interrupt()` to restore the flag — otherwise you silently swallow the cancellation signal and callers up the stack can't tell the thread was asked to stop.

Daemon vs user threads

```
Thread daemon = new Thread(() -> { while (true) { /* background housekeeping */ } });
daemon.setDaemon(true); // must be called BEFORE start()
daemon.start();
```

- **User threads** keep the JVM alive — the process won't exit until every user thread finishes.
- **Daemon threads** are background helpers (GC, timer ticks) — the JVM exits as soon as all *user* threads finish, **killing any still-running daemon threads mid-work**, no cleanup.
- `setDaemon(true)` must run before `start()`; calling it after throws `IllegalThreadStateException`.

Traps interviewers probe

- **"What happens if I call `run()` instead of `start()` ?"** No new thread — it just runs inline on the caller, defeating the whole point of threading. See above.
- **"What if I call `start()` twice on the same Thread object?"** Throws `IllegalThreadStateException` — a `Thread` object is single-use; once it leaves `NEW` it can never go back.
- **"You caught `InterruptedException` — then what?"** If they don't hear "restore the interrupt flag" or "rethrow," flag it — swallowing it silently is a common production bug that makes graceful shutdown impossible.
- **"Can you rely on the order threads run in?"** No — the scheduler is free to interleave `RUNNABLE` threads in any order; never assume thread A finishes before thread B without an explicit `join()`, lock, or other synchronization.
- **"What's a thread leak?"** Starting threads (directly or via an `ExecutorService` you forget to `shutdown()`) that never terminate and are never tracked — they pile up, hold memory/CPU, and can prevent JVM shutdown if they're user threads.
- **"Does `sleep()` release the lock it holds?"** No — `sleep()` just pauses; only blocking on another lock (or `wait()`) releases the monitor. Mixing this up with `wait()` is a common slip.

What to say (crisp version)

"I create tasks with `Runnable` or `Callable` rather than extending `Thread`, so the task isn't coupled to how it runs. A thread moves `NEW` → `RUNNABLE` → (`BLOCKED`/`WAITING`/`TIMED_WAITING`) → `TERMINATED`; `start()` is what actually spawns the OS thread — calling `run()` directly just executes inline on the caller, which is a classic bug. `join()` waits for a thread to finish, `sleep()` pauses without releasing locks, and `interrupt()` is a cooperative request — I always check `isInterrupted()` or handle `InterruptedException` by restoring the flag, otherwise cancellation silently gets lost. Daemon threads don't keep the JVM alive and can be killed mid-task on shutdown, so I only use them for background work I don't need to complete cleanly."

9. synchronized, volatile & the Java Memory Model

The concept in one line. Without help, threads may each work off a **stale cached copy** of a shared variable (a visibility bug) or **stomp on each other mid-update** (an atomicity bug); `volatile` fixes the first, `synchronized` fixes both.

The problem: caches and interleaving

Modern CPUs don't read/write main memory directly — each core keeps values in registers or a local cache. Two threads on two cores can each hold their own copy of the same variable.

```
main memory:  flag = false

CPU 1 (Thread A)      CPU 2 (Thread B)
+-----+             +-----+
| cache: flag=F |     | cache: flag=F |
+-----+             +-----+
    |                   |
    A writes flag=true   B still reads
    (may stay in A's     its own cached
    store buffer / cache  copy: flag=F
    line, not yet flushed) -> loop never exits
```

That's the **visibility** problem. There's a second, separate problem: **atomicity**. An operation like `count++` is really three steps — read, increment, write. If two threads interleave those steps, an increment can be lost:

```
count = 0
Thread A: read 0      Thread B: read 0
Thread A: add 1 -> 1   Thread B: add 1 -> 1
Thread A: write 1     Thread B: write 1  // A's increment is LOST
final count = 1      (expected 2)
```

Two different diseases, two different cures.

volatile — visibility (and ordering), not atomicity

`volatile` tells the JVM: never cache this field in a register/CPU-local cache across reads; every read goes to main memory, every write flushes to main memory immediately. It also stops the compiler/CPU from **reordering** instructions around that access.

```
without volatile:      with volatile:
write may sit in a     write is flushed to
store buffer -> other  main memory immediately
threads don't see it   other threads' reads see it
----->
```

Correct use: a stop flag

```
class Worker extends Thread {
    private volatile boolean running = true;    // visibility is all we need

    void stop() { running = false; }           // one thread writes

    @Override public void run() {
        while (running) {                      // another thread reads
            // do work
        }
    }
}
```

This works because `stop()` performs a single, independent write, and `run()` performs a single, independent read. No read-modify-write, so visibility alone is enough.

Incorrect use: a counter

```
class Counter {
    private volatile int count = 0;

    void increment() { count++; } // NOT atomic: read, add, write – 3 steps
}
```

`volatile` guarantees every thread sees the latest value before each step, but it does nothing to stop two threads from reading the same value and both writing `+1` from it. The classic mistake: "I made it `volatile` so it's thread-safe" — no, `volatile` never makes a compound operation atomic. Fixes: `synchronized`, or `AtomicInteger.incrementAndGet()` (a CAS loop under the hood — see the `Atomics/CAS` topic).

synchronized — mutual exclusion + visibility

`synchronized` uses an object's **intrinsic lock** (monitor). Only one thread can hold a given object's monitor at a time; a thread blocks at the door until the lock is free.

```
object's monitor (one lock per object)
Thread A -----> [ acquire ] --- runs block --- [ release ]
Thread B -----> waits -----> [ acquire ] --- runs
```

Entering a `synchronized` block/method **acquires** the lock; leaving it (normally or via exception) **releases** it. Critically, acquire/release also gives you visibility: everything Thread A wrote before releasing is guaranteed visible to Thread B after B acquires the *same* lock.

```

class Counter {
    private int count = 0;                // does NOT need to be volatile

    public synchronized void increment() { // acquire lock -> ++ -> release lock
        count++;
    }

    public synchronized int get() {       // must also synchronize the READ
        return count;
    }
}

```

Note the reader (`get()`) is synchronized too — without that, a reading thread has no happens-before edge to the writer and could still see a stale `count` .

Method vs. block, and what you lock on

```

synchronized void method() { ... }    // locks on "this" for the whole method
static synchronized void s() { ... }  // locks on the Class object

void method() {
    synchronized (this) { ... }        // block form: only lock the critical part
}

```

Locking on `this` exposes your lock to any other code that also happens to synchronize on your object — a caller could accidentally (or maliciously) grab your lock and stall you. Prefer a **private final lock object** dedicated to that purpose:

```

class Account {
    private final Object lock = new Object();
    private int balance;

    void withdraw(int amt) {
        synchronized (lock) {           // nobody outside this class can reach "lock"
            balance -= amt;
        }
    }
}

```

happens-before — the rule that makes all of this precise

The **Java Memory Model** defines a partial order called **happens-before**. Informally:

If action A happens-before action B, then everything A wrote is guaranteed visible to B, and A's effects appear to have completed before B runs.

Without a happens-before edge between two threads' accesses to the same variable, the JVM is free to let them see different, stale, or reordered values — that's the whole `volatile` / `synchronized` bug class in one sentence.

Thread A		Thread B	
write x = 42			
unlock(L)	----->	lock(L)	<- happens-before edge
		read x	-> guaranteed to see 42

Key happens-before edges to know cold:

- **Monitor unlock → later lock** on the *same* monitor (this is what makes `synchronized` work — release in A, acquire in B).
- **volatile write → later volatile read** of the *same* field.
- **Thread.start()** happens-before anything the started thread does.
- **Everything a thread does** happens-before another thread's successful **Thread.join()** on it.
- Program order within a single thread (trivially, each action happens-before the next).

No edge = no guarantee, even if in practice it "usually works" on your laptop.

Traps interviewers probe

- **"Is `volatile int count; count++;` thread-safe?"** No — visibility isn't atomicity. Read, add, write is 3 ops; use `synchronized` or `AtomicInteger`.
- **"What can I synchronize on?"** Any object, but never on a mutable field, a boxed type (`Integer`), or a `String` literal — boxed values can be shared/cached (`Integer` caches -128..127) and `String` literals are interned, so two unrelated pieces of code can accidentally lock on the *same* object, or a reassigned field means different threads lock on *different* objects, giving you no real mutual exclusion. Use a dedicated `private final Object lock`.
- **"Does double-checked locking work without `volatile`?"** No — this is the singleton double-checked-locking bug: without `volatile` on the instance field, a thread can observe a partially-constructed object due to reordering. Covered in depth in the Singleton topic.
- **"Why not just synchronize everything?"** Correctness, but at a cost: every extra `synchronized` block is a potential contention point — threads queue up for the lock, killing throughput. Lock only the critical section, keep it short, and prefer `java.util.concurrent` utilities (`ConcurrentHashMap` , atomics) which use finer-grained or lock-free strategies.

What to say (crisp version)

"Shared mutable state has two separate hazards: visibility, because CPUs cache values locally, and atomicity, because compound operations aren't a single step. `volatile` solves visibility and ordering by forcing every read/write through main memory, but `count++` under `volatile` still isn't safe because it's read-modify-write. `synchronized` solves both: it gives mutual exclusion via the object's monitor, and the JMM guarantees a happens-before edge from unlock to the next lock, so writes before release are visible after acquire. I lock on a private final object, not `this` or a `String`/boxed field, and I keep `synchronized` blocks as small as possible to avoid contention."

10. Locks & Atomic Variables

The concept in one line. When `synchronized` isn't flexible enough, `ReentrantLock` and friends give you explicit control over locking; when you only need to protect a *single* variable, atomics give you thread safety **without locking at all**, via CAS.

ReentrantLock vs synchronized

`synchronized` is a block-scoped lock: the JVM acquires it on entry and *always* releases it on exit (even via exception). `ReentrantLock` gives you the same mutual exclusion, but as an object you call `lock()` / `unlock()` on yourself — which means you must remember to release it, and you must do it in a `finally`.

```
synchronized(obj) { ... }  
  
JVM guarantees release  
even on exception
```

```
lock.lock();  
try {  
    ...  
} finally {  
    lock.unlock();    // MUST be here  
}
```

```
private final ReentrantLock lock = new ReentrantLock();  
  
void transfer() {  
    lock.lock();           // blocks until acquired  
    try {  
        // critical section  
    } finally {  
        lock.unlock();     // guaranteed to run, even on exception  
    }  
}
```

Why bother when `synchronized` is simpler? `ReentrantLock` adds capabilities `synchronized` doesn't have:

- **`tryLock()` / `tryLock(timeout, unit)`** — attempt to acquire without blocking forever; back off and do something else if the lock is busy. `synchronized` can only block.
- **`lockInterruptibly()`** — a thread blocked waiting for the lock can be interrupted and bail out, instead of waiting indefinitely.
- **Fairness** — `new ReentrantLock(true)` grants the lock in roughly the order threads requested it (FIFO), avoiding starvation. Default (`false`) is unfair but faster.
- **Both are reentrant** — a thread already holding the lock can re-acquire it (e.g. a recursive call) without deadlocking itself.

```

if (lock.tryLock(500, TimeUnit.MILLISECONDS)) {
    try {
        // got it in time
    } finally {
        lock.unlock();
    }
} else {
    // couldn't acquire – do something else instead of blocking forever
}

```

ReentrantReadWriteLock — many readers OR one writer

Most data is read far more often than it's written. Guarding it with a plain mutex forces readers to serialize even though reading concurrently is perfectly safe. `ReentrantReadWriteLock` splits the lock into two: a **read lock** that many threads can hold at once, and a **write lock** that is fully exclusive.

Read-heavy access pattern:

readLock held by: T1 T2 T3 T4 <- all run concurrently

 writeLock held by: T5 <- alone; no readers, no other writer

Rule: (many readers) OR (exactly one writer) – never both together

```

private final ReentrantReadWriteLock rw = new ReentrantReadWriteLock();
private final Lock readLock = rw.readLock();
private final Lock writeLock = rw.writeLock();
private final Map<String, String> cache = new HashMap<>();

String get(String key) {
    readLock.lock();
    try {
        return cache.get(key);
    } finally {
        readLock.unlock();
    }
}

void put(String key, String value){
    writeLock.lock();
    try {
        cache.put(key, value);
    } finally {
        writeLock.unlock();
    }
}

```

This helps when reads vastly outnumber writes (a config cache, a routing table) — readers no longer block each other. If writes are frequent, the bookkeeping overhead can make it *slower* than a plain lock, so it's not a free win.

Atomics — lock-free via CAS

`AtomicInteger`, `AtomicLong`, `AtomicReference` let multiple threads update a single variable correctly with **no lock at all**. Under the hood they use a CPU instruction called **compare-and-swap (CAS)**: "update this variable to `newValue`, but only if it still equals `expected`; otherwise tell me you failed."

```
CAS(location, expected, newValue):
```

```
    read current value --> is it == expected? --+--yes--> write newValue, return true
                                           |
                                           +--no----> return false (someone else won)
```

Typical usage – retry loop:

```
loop:
    old = get()
    new = compute(old)
    if CAS(old, new) succeeds -> done
    else                        -> retry (old changed, recompute and try again)
```

No thread ever blocks; a losing thread just retries with the fresh value. That's why atomics are called "lock-free" — progress doesn't depend on the OS scheduler handing out a lock.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();           // atomic ++counter, CAS loop under the hood

// Manual CAS loop – e.g. atomic "multiply by 2, but never exceed 100"
AtomicInteger value = new AtomicInteger(10);
int old, updated;
do {
    old = value.get();
    updated = Math.min(old * 2, 100);
} while (!value.compareAndSet(old, updated));
```

`incrementAndGet()`, `addAndGet()`, `updateAndGet()` are all just this same retry loop, done for you.

When to use what

Need	Reach for
One counter / flag / reference updated by many threads	Atomic* (fastest, no blocking)
Multiple fields that must change together, consistently	Lock (<code>synchronized</code> or <code>ReentrantLock</code>)
Simple, low-contention mutual exclusion	synchronized (simplest, least error-prone)
Need <code>tryLock</code> , fairness, or interruptible waits	ReentrantLock
Read-heavy shared state	ReentrantReadWriteLock
Very high-contention counter (many threads incrementing)	LongAdder

LongAdder, briefly: under heavy contention, every thread hammering one `AtomicLong` still fights over the *same* CAS location and retries pile up. `LongAdder` internally keeps a striped array of counters — different threads update different cells, and the total is only summed when you call `sum()`. Trade-off:

faster under contention, but reading the running total is not as instantaneous/consistent as `AtomicLong.get()`. Use it for hot counters (metrics, request counts) where you sum occasionally rather than read every increment.

Worked example — the composition trap

```
AtomicInteger balance = new AtomicInteger(100);
AtomicInteger withdrawals = new AtomicInteger(0);

// WRONG: each atomic op is individually atomic, but the PAIR is not.
void withdraw(int amount) {
    balance.addAndGet(-amount);    // atomic on its own
    withdrawals.incrementAndGet(); // atomic on its own
    // another thread can observe balance updated but withdrawals not yet,
    // or read balance/withdrawals out of sync with each other
}

// RIGHT: compound invariant across two fields -> needs a lock
private final ReentrantLock lock = new ReentrantLock();
void withdrawSafely(int amount) {
    lock.lock();
    try {
        balance.addAndGet(-amount);
        withdrawals.incrementAndGet();
    } finally {
        lock.unlock();
    }
}
```

Traps interviewers probe

- **"You forgot to unlock — what happens?"** With `ReentrantLock`, if `unlock()` isn't in a `finally` and an exception is thrown between `lock()` and `unlock()`, the lock is **never released** — every other thread blocks forever. This is the #1 reason `synchronized` is often preferred: the JVM can't let you make this mistake.
- **"Are two atomics together atomic?"** No — see the worked example above. Each `AtomicInteger` op is atomic *by itself*, but composing two of them is not a single atomic operation. If invariants span multiple variables, use a lock.
- **"What's the ABA problem?"** A thread reads value `A`, gets preempted; another thread changes it `A -> B -> A`; the first thread's CAS sees `A` again and succeeds, even though the value changed in between — which can be wrong if intermediate state mattered (e.g. a reused node in a lock-free stack). Plain CAS can't detect this because it only compares the *value*. Fix: `AtomicStampedReference`, which pairs the value with a version stamp that must also match, so an `A→B→A` round trip is detectable.
- **"Multiple locks, multiple threads, opposite order?"** That's lock-ordering deadlock — two threads each hold one lock and wait on the other's. Covered in the deadlock/liveness topic; the fix is a consistent global lock-acquisition order.
- **"Why not just use `synchronized` everywhere?"** You can, for simple cases — it's harder to misuse. Reach for `ReentrantLock` specifically when you need `tryLock`, timeouts, interruptible waits, or fairness; reach for atomics when the state really is just one variable.

What to say (crisp version)

"ReentrantLock is synchronized with more control — tryLock, timeouts, interruptibility, fairness — but you must unlock in a finally or you leak the lock forever.

ReentrantReadWriteLock lets many readers run concurrently and only excludes everyone during a write, which helps when reads dominate. Atomics avoid locking entirely using CAS — compare-and-swap: try to update, and if another thread got there first, retry with the new value. They're great for single counters, but two atomics updated together aren't atomic as a pair — that needs a lock. For very hot counters I'd use LongAdder over AtomicLong to reduce CAS contention."

11. Producer–Consumer Pattern

The concept in one line. Producers and consumers talk through a **bounded shared buffer**: producers block when it's full, consumers block when it's empty, and the buffer's capacity keeps memory bounded and the two sides working at a sustainable pace.

The problem

```
BOUNDED BUFFER (capacity = 4)

Producer(s) --put()--> [ x | x | x |   ] --take()--> Consumer(s)
                        full? BLOCK      empty? BLOCK

- buffer FULL   -> put() blocks until a consumer takes something
- buffer EMPTY  -> take() blocks until a producer puts something
- buffer has room & items -> both proceed without blocking
```

Without coordination you get two failure modes: an **unbounded** queue that grows forever under a slow consumer (OOM), or a race where a consumer reads from an empty buffer / a producer overwrites data that hasn't been read yet. The classic fix needs three things: mutual exclusion on the buffer, a way to block "buffer full", and a way to block "buffer empty" — then wake the other side up when state changes.

Solution 1 (recommended): `BlockingQueue`

`ArrayBlockingQueue` (or `LinkedBlockingQueue`) already implements the bounded buffer with internal locking and condition variables. `put()` blocks while full, `take()` blocks while empty — you never touch `wait / notify` yourself.

```
ArrayBlockingQueue(capacity=4)
internally: 1 lock + 2 Conditions ("notFull", "notEmpty")

put() -> lock -> while full: notFull.await()
        -> enqueue -> notEmpty.signal() -> unlock

take() -> lock -> while empty: notEmpty.await()
        -> dequeue -> notFull.signal() -> unlock
```

Termination is the part people forget: workers block forever in `take()` unless you tell them to stop. The cleanest way is a **poison pill** — a sentinel value that means "no more work" — one per consumer thread.

```

import java.util.concurrent.*;

public class ProducerConsumerBlockingQueue {

    private static final String POISON_PILL = "__STOP__";

    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<String> buffer = new ArrayBlockingQueue<>(4); // bounded!
        int numProducers = 2;
        int numConsumers = 3;

        ExecutorService pool = Executors.newFixedThreadPool(numProducers + numConsumers);

        // Producers
        for (int p = 0; p < numProducers; p++) {
            int id = p;
            pool.submit(() -> {
                try {
                    for (int i = 0; i < 5; i++) {
                        String item = "P" + id + "-item" + i;
                        buffer.put(item); // blocks if full
                        System.out.println("produced " + item);
                    }
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }

        // Consumers
        for (int c = 0; c < numConsumers; c++) {
            pool.submit(() -> {
                try {
                    while (true) {
                        String item = buffer.take(); // blocks if empty
                        if (item.equals(POISON_PILL)) break;
                        System.out.println(" consumed " + item);
                    }
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }

        pool.shutdown();
        // In real code: wait for producers to finish, THEN feed one pill per consumer.
        // (Simplified here – a CountdownLatch on producers is the clean way to sequence this
        for (int c = 0; c < numConsumers; c++) {
            buffer.put(POISON_PILL);
        }
        pool.awaitTermination(5, TimeUnit.SECONDS);
    }
}

```

Why this is the "recommended" answer: no manual locking, no risk of a lost wakeup, and the bounded capacity gives you natural **backpressure** — a slow consumer automatically throttles fast producers instead of letting memory balloon.

Solution 2: `wait()` / `notifyAll()` on a shared buffer

This is the version that proves you understand what `BlockingQueue` does under the hood. Every access is inside a `synchronized` block (the monitor), and you park on `wait()` instead of spinning.

```
synchronized(lock) {
    while (buffer.full) lock.wait();    // producer parks, releases lock
    ... add item ...
    lock.notifyAll();                  // wake any waiting consumer
}
```

```
import java.util.LinkedList;
import java.util.Queue;

public class BoundedBuffer<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    public BoundedBuffer(int capacity) {
        this.capacity = capacity;
    }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) {    // WHILE, not if
            wait();                          // releases the monitor, parks here
        }
        queue.add(item);
        notifyAll();                          // wake any thread waiting on this monitor
    }

    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) {             // WHILE, not if
            wait();
        }
        T item = queue.poll();
        notifyAll();
        return item;
    }
}
```

```

public class ProducerConsumerWaitNotify {
    public static void main(String[] args) {
        BoundedBuffer<Integer> buffer = new BoundedBuffer<>(4);

        Runnable producer = () -> {
            try {
                for (int i = 0; i < 10; i++) {
                    buffer.put(i);
                    System.out.println("produced " + i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        };

        Runnable consumer = () -> {
            try {
                for (int i = 0; i < 10; i++) {
                    int item = buffer.take();
                    System.out.println(" consumed " + item);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        };

        new Thread(producer).start();
        new Thread(consumer).start();
    }
}

```

Why `while`, never `if`. Two independent reasons the thread can wake up without the condition actually being true:

1. **SPURIOUS WAKEUP** – the JVM/JLS explicitly allows `wait()` to return with no `notify()` at all. ``if`` would proceed on a false premise.
2. **STOLEN CONDITION** – `notifyAll()` wakes 3 waiting consumers, but only 1 item was added. Consumer A re-acquires the lock and takes it; Consumers B and C wake up too, re-check, buffer is empty again -> they **MUST** re-wait. ``if`` would let B or C dequeue from an empty queue.

`wait()` gives no guarantee about *what's true* when it returns — only that the monitor was re-acquired. The only safe pattern is: **wake up, re-check the condition yourself, loop back to `wait()` if it's still false**. That's exactly what `while` does.

Traps interviewers probe

- **`if (empty) wait();` instead of `while`**. Works "most of the time" in testing, then fails under load — a classic heisenbug. Always loop.
- **`notify()` vs `notifyAll()` — the lost wakeup**. `notify()` wakes *one* arbitrary waiting thread. If you have both producers and consumers waiting on the *same* monitor and you `notify()` a thread that can't actually proceed (e.g. you woke another producer instead of a consumer), the thread that could make progress may never get woken — a **lost wakeup**, and the system

deadlocks. `notifyAll()` is the safe default; combined with `while`, only the threads that find the condition true actually proceed, others just re-wait cheaply.

- **Calling `wait()` / `notify()` outside a `synchronized` block on that object.** Throws `IllegalMonitorStateException` — you must hold the monitor to call either. It's also *why* they must be called on the same object you synchronized on.
- **Unbounded queue → OOM.** A `LinkedList` used as a queue with no capacity check, or a `LinkedBlockingQueue` constructed without a capacity, will happily grow forever if producers outrun consumers. Always bound it, or you've traded a deadlock for a slow OOM.
- **Forgetting termination.** Consumers blocked in `take()` / `wait()` never exit on their own. Need a poison pill, a shared "shutdown" flag checked in the loop condition, or interrupting the threads and handling `InterruptedException` as the exit signal.
- **"Why not just poll in a loop with `sleep()` ?"** Wastes CPU (busy-waiting) and adds latency. `wait()` / `take()` block efficiently and wake immediately on notification.

What to say (crisp version)

"Producer-consumer needs a bounded buffer, blocking when full or empty. In practice I reach for `BlockingQueue` — `put()` / `take()` already handle the locking and blocking, and the bound gives backpressure for free; termination is a poison pill or a shutdown flag. If asked to build it from scratch, I use `synchronized` plus `wait()` / `notifyAll()`, and the one non-negotiable rule is `wait()` inside a `while` loop, never `if` — spurious wakeups and `notifyAll()` waking threads whose condition isn't actually true both require re-checking after wait returns. And `notifyAll()` over `notify()` unless I can prove any woken thread can make progress, or I risk a lost wakeup."

12. Thread-Safe Singleton

The concept in one line. A singleton must hand out **exactly one instance** of a class, and under concurrency that guarantee has to hold even when many threads call `getInstance()` at the same instant — which is where most "write it on the whiteboard" answers break.

The requirement

Requirement:

- Only ONE instance of the class ever exists in the JVM
- Safe when N threads call `getInstance()` concurrently, first time
- (ideally) lazy: don't pay construction cost if nobody asks

Attempt 1 — Eager initialization (static final)

```
public class Config {
    private static final Config INSTANCE = new Config();
    private Config() { }
    public static Config getInstance() { return INSTANCE; }
}
```

- **Thread-safe:** yes — the JVM class-loading mechanism initializes static fields once, before any thread can observe `Config.class`, guarded by an implicit lock.
- **Downside:** `INSTANCE` is created the moment the class loads, even if `getInstance()` is never called — wasteful if construction is expensive or has side effects.

Attempt 2 — Naive lazy `getInstance()` — BROKEN

```
public class Config {
    private static Config instance;
    private Config() { }
    public static Config getInstance() {
        if (instance == null) {           // <-- check
            instance = new Config();      // <-- create
        }
        return instance;
    }
}
```

This is the classic "spot the bug" trap. Two threads can both pass the null check before either finishes constructing:

Thread A	Thread B
-----	-----
read instance -> null	
	read instance -> null
new Config() ...	
instance = A's object	
	new Config() ...
	instance = B's object <--- overwrites A's!

Result: two different Config objects were live at once,
and callers may have already handed out A's reference.

Nothing here is atomic — the read-null-check, the construction, and the write are three separate steps with a window between them. This is a **race condition**, not a rare edge case; under load it happens routinely.

Attempt 3 — synchronized getInstance() — correct but slow

```
public class Config {
    private static Config instance;
    private Config() { }
    public static synchronized Config getInstance() {
        if (instance == null) {
            instance = new Config();
        }
        return instance;
    }
}
```

- **Correct:** the whole check-then-create is now one atomic critical section.
- **Problem:** *every single call* — forever, even the 10-millionth call long after `instance` is set — acquires the lock. That's needless contention for a value that never changes again after the first assignment.

```
getInstance() calls over time:
[LOCK] [LOCK] [LOCK] [LOCK] [LOCK]... <- lock taken every time, unnecessary after warm-up
```

Attempt 4 — Double-Checked Locking (DCL) with `volatile`

The fix: only synchronize on the *first* call, by checking for null once outside the lock and again inside it.


```

public class Config {
    private static volatile Config instance;    // volatile is NOT optional – see below
    private Config() { }

    public static Config getInstance() {
        if (instance == null) {                // 1st check, no lock (fast path)
            synchronized (Config.class) {
                if (instance == null) {        // 2nd check, under lock
                    instance = new Config();
                }
            }
        }
        return instance;
    }
}

```

Fast path (instance already set):
 if (instance == null) -> false
 return instance
 (no lock ever taken)

Slow path (first callers, racing):
 if (instance == null) -> true
 synchronized { ... check again ... }
 only ONE thread actually constructs it

Why `volatile` is required — the key insight. `new Config()` is not one atomic step to the JVM/CPU. Conceptually it's three:

1. allocate memory for the object
2. run the constructor (initialize its fields)
3. assign the reference into `instance`

Without `volatile`, the compiler/JIT/CPU are allowed to **reorder steps 2 and 3** (there's no happens-before edge forcing step 2 to finish before step 3 is visible to other threads). So another thread can see `instance` as **non-null but pointing at a half-built object** — its fields still at default values (0, null, false):

Thread A (constructing)	Thread B (reading, no volatile)
-----	-----
instance = <allocated, unset fields>	// reorder exposes the reference early
	if (instance == null) -> false, skip lock
	return instance.someField // reads 0 / null !
(constructor body still running)	

`volatile` inserts the necessary memory barriers so that everything written inside the constructor (step 2) is visible to any thread that observes the non-null reference (step 3) — it restores the happens-before ordering. This is the single most-tested detail of DCL: **DCL without `volatile` is still broken**, just broken in a subtler, harder-to-reproduce way than Attempt 2.

Attempt 5 — Bill Pugh / static holder idiom (recommended lazy option)

```
public class Config {
    private Config() { }

    private static class Holder {          // not loaded until referenced
        static final Config INSTANCE = new Config();
    }

    public static Config getInstance() {
        return Holder.INSTANCE;           // no lock, no volatile, no DCL
    }
}
```

- **Lazy:** the nested `Holder` class is only loaded (and its static field initialized) the first time `getInstance()` runs and touches `Holder.INSTANCE`.
- **Thread-safe:** the JVM classloader itself guarantees a class is initialized exactly once, synchronized under the hood — the same guarantee Attempt 1 relies on, just deferred.
- **Fast:** after the first call, it's a plain static field read — no lock, no `volatile` check, nothing to get wrong. This is the idiom to reach for when asked for "the best lazy thread-safe singleton in plain Java."

Attempt 6 — enum singleton

```
public enum Config {
    INSTANCE;
    void doWork() { /* ... */ }
}
// usage: Config.INSTANCE.doWork();
```

Effective Java's favorite: a single-element enum. The JVM guarantees enum constants are constructed exactly once, thread-safely, and — unlike the other approaches — it's **immune to reflection-based re-instantiation and to breaking under deserialization** for free, because enum serialization/deserialization is handled specially by the JVM (it resolves by name to the existing constant rather than calling a constructor).

Traps interviewers probe

- **"Write me a thread-safe lazy singleton" then "why is `volatile` there?"** If you can't explain the reordering (allocate/construct/assign) argument above, that's the tell you memorized DCL without understanding it.
- **Reflection can break "safe" singletons.** `Constructor.setAccessible(true)` can call a `private` constructor a second time, producing a second instance, even for Attempt 1/4/5. Guarding against it: throw in the constructor if `instance != null`. Enum singletons are naturally immune — you cannot reflectively invoke an enum's constructor to create a second constant.
- **Serialization can break it too.** Deserializing a singleton with plain `readObject` / default serialization creates a brand-new object. Fix: implement `readResolve()` to return the existing instance. Enum again sidesteps this automatically.

- **Singleton as a testability smell.** Beyond the concurrency puzzle, interviewers may push on design: a singleton is global mutable state — it hides dependencies, makes unit tests hard to isolate (state leaks between tests), and resists mocking. The usual answer: prefer dependency injection with a singleton-scoped bean (Spring `@Scope("singleton")` , Guice `@Singleton`) over a hand-rolled `getInstance()` , so the container owns the lifecycle and tests can substitute a fresh instance.

What to say (crisp version)

"The naive lazy check-then-create races: two threads can both see null and construct two instances. `synchronized` on every call fixes it but is slow forever. Double-checked locking fixes the slowness by only locking on the first call — but the field must be `volatile` , because `new` isn't atomic: without a memory barrier another thread can see a non-null reference to an object whose constructor hasn't finished running, due to reordering of allocation and field assignment. In plain Java I'd actually reach for the Bill Pugh static-holder idiom — the classloader gives you lazy, thread-safe, lock-free initialization for free — or an enum singleton, which also closes the reflection and serialization loopholes the other forms have."

13. Executors & Thread Pools

The concept in one line. Creating a raw `new Thread()` per task is expensive and unbounded; an `ExecutorService` maintains a **pool of reusable worker threads** plus a **queue of pending tasks**, decoupling "submit work" from "how/when it runs."

Why not `new Thread()` everywhere

<code>new Thread()</code> per task:	thread pool:
task1 -> [Thread] (created, run, destroyed)	task1 -\
task2 -> [Thread] (created, run, destroyed)	task2 --> [pool of N reusable workers]
task3 -> [Thread] ...	task3 -/ (created once, parked between tasks, never destroyed)
-> OS thread creation/teardown cost paid every single time	-> pay creation cost ONCE, reuse workers
	-> no cap = unbounded threads = OOM/ thread-thrashing under load

- Every OS thread costs memory (its stack, typically ~512KB-1MB by default) and CPU time to create/schedule/tear down. Spinning up a new one per request doesn't scale.
- Uncontrolled thread creation is **unbounded** — a burst of requests can spawn thousands of threads, thrashing the scheduler and exhausting memory before any of them finish real work.
- A pool amortizes creation cost across many tasks and caps concurrency to something the machine can actually handle.

ExecutorService and the Executors factory

```
ExecutorService fixed   = Executors.newFixedThreadPool(4);           // N threads, unbounded queue
ExecutorService cached  = Executors.newCachedThreadPool();          // grows unbounded, reuses idle threads
ExecutorService single  = Executors.newSingleThreadExecutor();       // 1 thread, tasks run in order
ScheduledExecutorService sched = Executors.newScheduledThreadPool(2); // delayed/periodic tasks
```

- `newFixedThreadPool(n)` — exactly `n` threads alive; extra tasks wait in an unbounded `LinkedBlockingQueue`.
- `newCachedThreadPool()` — creates threads on demand, reuses idle ones for 60s, but has **no cap** — under sustained load it can create unbounded threads.
- `newSingleThreadExecutor()` — one worker, guarantees tasks run sequentially in submission order (useful when you need serialized access without manual locking).
- `newScheduledThreadPool(n)` — like fixed, but supports `schedule()`, `scheduleAtFixedRate()`, `scheduleWithFixedDelay()` for delayed/recurring work.

In production, prefer a hand-configured `ThreadPoolExecutor` instead of the `Executors` factory methods, so you control every knob explicitly (see the trap below on why).

ThreadPoolExecutor anatomy

```
submit(task) ----> [ bounded task QUEUE ]
                    |
                    | queue full? and pool < max? -> spawn a new worker
                    |
                    v
                    +-----+
                    | worker1 | worker2 | ... up to maxPoolSize
                    +-----+
                    (idle beyond corePoolSize for keepAliveTime -> thread dies)

queue FULL and pool AT maxPoolSize -> RejectionPolicy kicks in
```

```
ExecutorService pool = new ThreadPoolExecutor(
    4,                      // corePoolSize - threads kept alive even when idle
    16,                     // maximumPoolSize - hard cap on worker threads
    60L, TimeUnit.SECONDS,  // keepAliveTime - how long extra threads idle before
    new ArrayBlockingQueue<>(100), // work queue - BOUNDED, so we fail fast instead
    new ThreadPoolExecutor.CallerRunsPolicy() // rejection policy when queue AND pool are full
);
```

- **corePoolSize** — minimum threads kept warm, even idle (unless `allowCoreThreadTimeOut` is set).
- **maximumPoolSize** — never create more workers than this, no matter how deep the queue is.
- **queue** — holds tasks once `corePoolSize` workers are busy; only once the queue is **full** does the pool grow toward `maximumPoolSize`.
- **keepAliveTime** — how long a thread above `corePoolSize` sits idle before being retired.
- **rejection policy** — what happens when the queue is full *and* the pool is at max: `AbortPolicy` (throws), `CallerRunsPolicy` (caller thread runs it, throttling the submitter), `DiscardPolicy` (silently drops), `DiscardOldestPolicy` (drops the oldest queued task).

submit vs execute, Future, invokeAll

```
ExecutorService pool = Executors.newFixedThreadPool(4);

pool.execute(() -> System.out.println("fire and forget")); // Runnable, no result, no Future

Future<Integer> f = pool.submit(() -> 40 + 2);              // Runnable OR Callable, return
Integer result = f.get();                                   // BLOCKS until done (or throws
Integer withTimeout = f.get(2, TimeUnit.SECONDS);           // bounded wait -> throws TimeoutException

List<Callable<Integer>> jobs = List.of(() -> 1, () -> 2, () -> 3);
List<Future<Integer>> results = pool.invokeAll(jobs);         // blocks until ALL complete
```

- **execute(Runnable)** — no return value, exceptions thrown inside the task are lost (printed to the uncaught exception handler, not visible to the caller).
- **submit(...)** — always returns a `Future`, so exceptions surface when you call `get()` (wrapped in `ExecutionException`), and results/cancellation are possible.
- **Future.get()** — blocks the calling thread indefinitely by default; always prefer the timed overload in production so a stuck task can't hang your caller forever.

- `invokeAll(...)` — submits a batch and blocks until every task finishes (or is cancelled); `invokeAny(...)` is the sibling that returns as soon as **one** succeeds.

Shutting down cleanly

```
pool.shutdown(); // stop accepting NEW tasks; let queued on
try {
    if (!pool.awaitTermination(30, TimeUnit.SECONDS)) {
        pool.shutdownNow(); // interrupts running tasks, drains the qu
    }
} catch (InterruptedException e) {
    pool.shutdownNow();
    Thread.currentThread().interrupt();
}
```

<code>shutdown():</code>	<code>shutdownNow():</code>
reject new tasks	reject new tasks
let queued + running tasks finish	INTERRUPT running tasks
-> graceful drain	return list of tasks never started
	-> fast, but tasks must handle interrupt

- Threads in a pool are **not daemon threads** by default — a live `ExecutorService` you never shut down will keep the JVM running forever.
- `awaitTermination(timeout, unit)` blocks until either everything finishes or the timeout elapses — pair it with `shutdownNow()` as a fallback, the standard idiom above.

Traps interviewers probe

- **"What's wrong with `Executors.newFixedThreadPool(n)` in prod?"** Its backing queue is an **unbounded** `LinkedBlockingQueue` — if tasks arrive faster than they're processed, the queue grows without limit and you OOM instead of failing fast. Same story for `newSingleThreadExecutor`. A hand-built `ThreadPoolExecutor` with a **bounded** queue and an explicit rejection policy fails predictably instead of silently consuming all memory.
- **"What if you forget to `shutdown()` the pool?"** Non-daemon worker threads stay alive forever, so the JVM never exits (relevant for CLI tools/tests; less visible in long-running servers, but still a resource leak if you keep creating pools).
- **"Is `future.get()` safe to call as-is?"** Only if you're certain the task terminates — otherwise it blocks forever. Always use the timeout overload or design tasks to be cancellable.
- **"What happens if a task in the pool blocks (e.g., a slow network call)?"** It occupies a worker thread for the duration. With a small fixed pool, enough blocking tasks can **starve** the pool — no threads left for other work even though the CPU is idle. Fix: size the pool for the workload (I/O-bound needs more threads than CPU-bound), use timeouts, or use a separate pool for blocking work.
- **"`execute` vs `submit` — does it matter which you use for a `Runnable`?"** Yes: with `execute`, an uncaught exception is lost (goes to the thread's uncaught-exception handler); with `submit`, it's captured and rethrown from `Future.get()` — much easier to notice bugs.

What to say (crisp version)

"Thread pools amortize the cost of thread creation and cap concurrency instead of spawning unbounded raw threads. I use `ExecutorService`, but in production I configure a `ThreadPoolExecutor` explicitly — core size, max size, a *bounded* queue, and a rejection policy — rather than `Executors.newFixedThreadPool`, because its default queue is unbounded and can OOM under load. `submit()` gives me a `Future` so exceptions and results are visible; I always call `get()` with a timeout. And I always shut the pool down — `shutdown()` to drain gracefully, `shutdownNow()` as a forced fallback after `awaitTermination` — otherwise the threads are non-daemon and keep the JVM alive."

14. Coordination: Latches, Barriers, Semaphores & Deadlock

The concept in one line. `CountDownLatch`, `CyclicBarrier`, and `Semaphore` are the standard `java.util.concurrent` tools for making threads **wait for each other** or **share a limited resource**, and misusing locks across them is exactly how you end up in a **deadlock**.

CountDownLatch — wait for N events, once

```
CountDownLatch latch = new CountDownLatch(3);    // count starts at 3

Runnable worker = () -> {
    // do setup work
    latch.countDown();                          // decrement; never blocks
};
for (int i = 0; i < 3; i++) new Thread(worker).start();

latch.await();                                // main blocks until count reaches 0
System.out.println("all 3 workers finished setup");
```

```
latch count = 3
worker1 -> countDown() -> count=2
worker2 -> countDown() -> count=1
worker3 -> countDown() -> count=0 -----> await() unblocks (ALL waiters released)

latch is now spent -> count can never go back up -> NOT reusable
```

- One-shot: once the count hits zero, the latch is permanently open — calling `countDown()` again does nothing, and you cannot reset it.
- Classic uses: wait for N services to finish initializing before starting traffic; wait for N parallel subtasks to finish before aggregating results.

CyclicBarrier — N threads wait for EACH OTHER, repeatedly

```
CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("all arrived, proceeding"))

Runnable phase = () -> {
    // do phase-1 work
    try {
        barrier.await();                // block until all 3 parties reach the barrier
    } catch (InterruptedException | BrokenBarrierException e) { /* handle */ }
    // do phase-2 work, could await() again -> barrier resets and can be reused
};
```


CountDownLatch:
1 event source counts down
N threads can await
fires once, done forever

CyclicBarrier:
N peer threads wait for EACH OTHER
exactly N parties must call await()
resets automatically -> reusable across "rounds"

- The barrier's count **resets** after all parties arrive, so the same `CyclicBarrier` can gate multiple rounds (e.g., a simulation where all threads must finish generation k before anyone starts generation $k+1$).
- Optional constructor `Runnable` runs once per round, on one of the arriving threads, right when the barrier trips — handy for aggregating/logging between rounds.

Semaphore — limit concurrent access to N permits

```
Semaphore semaphore = new Semaphore(3);    // only 3 concurrent permits

void accessResource() throws InterruptedException {
    semaphore.acquire();                    // blocks if no permit available
    try {
        // use the limited resource (e.g., a connection pool slot)
    } finally {
        semaphore.release();                // MUST release, even on exception
    }
}
```

```
permits: [*][*][*] (3 available)
T1 acquire() -> [ ][*][*]      2 left
T2 acquire() -> [ ][ ][*]      1 left
T3 acquire() -> [ ][ ][ ]      0 left
T4 acquire() -> BLOCKS until someone release()
```

- Not tied to a specific "owner" thread the way a lock is — any thread can `release()`, even one that never called `acquire()` (useful, but also a footgun if misused).
- Classic uses: capping concurrent DB connections, limiting parallel outbound API calls, bounding how many threads can enter a section at once (a more flexible `synchronized`).

Deadlock — the four Coffman conditions

A deadlock needs **all four** of these simultaneously:

1. **Mutual exclusion** — resources (locks) can't be shared.
2. **Hold and wait** — a thread holds one lock while waiting for another.
3. **No preemption** — a lock can't be forcibly taken away from the thread holding it.
4. **Circular wait** — a cycle of threads, each waiting on a lock the next one holds.

```

Thread A                                Thread B
lock(lockX)  ---- holds X ---->
                                lock(lockY)  ---- holds Y ---->
lock(lockY)  <---- WAITS for Y ---+
                                lock(lockX)  <---- WAITS for X ---+
                                *** both threads permanently blocked ***

      A holds X, wants Y  --\    /--- B holds Y, wants X
                          \  /
                          \ /
                          CIRCULAR WAIT

```

```

Object lockX = new Object(), lockY = new Object();

// Thread A                                // Thread B
synchronized (lockX) {                      synchronized (lockY) {
    synchronized (lockY) { ... }              synchronized (lockX) { ... }
}                                              }
// if these interleave badly, A holds X waiting for Y
// while B holds Y waiting for X -> deadlock

```

Prevention

- **Consistent lock ordering** — always acquire locks in the same global order (e.g., by a fixed ID) across every code path; breaks the circular-wait condition entirely.
- **tryLock(timeout)** (from `Lock / ReentrantLock`) — instead of blocking forever, give up and back off after a timeout, then retry; avoids permanent deadlock at the cost of needing retry logic.
- One-line mentions: **livelock** — threads keep actively changing state in response to each other but no one makes progress (e.g., both repeatedly backing off and retrying in lockstep); **starvation** — a thread never gets CPU time or a lock because other threads keep being favored (e.g., an unfair lock policy or higher-priority threads hogging resources).

Traps interviewers probe

- **"Can you reuse a `CountDownLatch`?"** No — once it hits zero it's done forever; reach for `CyclicBarrier` if you need repeated synchronization rounds.
- **"What happens if one party in a `CyclicBarrier` dies or times out?"** Every other thread waiting in `await()` gets a `BrokenBarrierException` — one failed party breaks the barrier for everyone, not just itself.
- **"What if you forget to `release()` a semaphore permit?"** A permit leak — over time available permits trend toward zero and everyone else blocks forever, even though the resource may actually be free. Always `release()` in a `finally` block.
- **"How do you actually debug/prevent a lock-ordering deadlock?"** Enforce a single global order for acquiring multiple locks everywhere in the codebase, or use `tryLock` with a timeout so threads back off instead of waiting forever. A thread dump showing two threads each `BLOCKED` waiting on a lock the other holds is the smoking gun.
- **"Is a deadlock the same as an infinite loop?"** No — deadlocked threads are `BLOCKED` (consuming no CPU, just parked), whereas livelocked threads are still actively running, burning CPU while making no real progress.

What to say (crisp version)

" `CountDownLatch` is one-shot — N events count down to zero and every waiter is released, but it can't be reset. `CyclicBarrier` is the reusable version — N peer threads wait for each other, and the barrier resets automatically for the next round. `Semaphore` caps how many threads can concurrently hold a resource via permits, and I always release in a `finally` to avoid leaking permits. Deadlock needs all four Coffman conditions — mutual exclusion, hold-and-wait, no preemption, and circular wait — so the practical fix is breaking circular wait: acquire locks in a consistent global order, or use `tryLock` with a timeout to back off instead of blocking forever."
